

Arquitectura por capas de FastKote

El sistema se organizó con una arquitectura por capas para separar responsabilidades y evitar que la interfaz, la lógica de negocio y la base de datos queden mezcladas.

1. Capa de presentación

Ubicación backend:

```
backend/src/interfaces/http
```

Responsabilidad:

- Recibir peticiones HTTP.
- Validar token JWT.
- Convertir datos de request a comandos o consultas.
- Enviar respuestas JSON.

Ejemplos:

- `auth.controller.ts`
- `clients.controller.ts`
- `employees.controller.ts`
- `quotes.controller.ts`

2. Capa de aplicación

Ubicación:

```
backend/src/application
```

Responsabilidad:

- Contener los casos de uso.

- Coordinar reglas del negocio.
- Usar Mediator para despachar comandos y consultas.
- Usar Strategy para cálculo de cotizaciones y control de agenda.

Ejemplos:

- `LoginUser.handler.ts`
- `CreateClient.handler.ts`
- `CreateQuote.handler.ts`
- `UpdateQuoteStatus.handler.ts`

3. Capa de dominio

Ubicación:

```
backend/src/domain
```

Responsabilidad:

- Definir contratos del negocio.
- Definir entidades base.
- Definir interfaces de repositorios.

Esta capa no depende de Express, Prisma ni PostgreSQL.

4. Capa de infraestructura

Ubicación:

```
backend/src/infrastructure
```

Responsabilidad:

- Implementar acceso real a PostgreSQL por medio de Prisma.
- Implementar generación de PDF.
- Implementar servicio de WhatsApp.

- Implementar hash de contraseñas y JWT.

5. Frontend

Ubicación:

frontend/src

Responsabilidad:

- Mostrar landing pública con carrusel, login, página de construcción para registro, clientes, empleados, cotizaciones y calendario.
- Consumir API REST.
- Mantener componentes de interfaz separados.
- Aplicar acceso visual por roles: Administrador y Empleado.

Flujo general

React Page -> API Client -> Express Controller -> Mediator -> Use Case -> Repository ->



Control de acceso por rol

La base de datos mantiene la relación normalizada `employees` -> `employee_roles` -> `roles` y `users` -> `employees` . El JWT incluye los roles del empleado activo. En el backend, los controladores usan `allowRoles(...)` para proteger módulos. En el frontend, `AppLayout` oculta el módulo de empleados cuando el usuario no tiene rol `Admin` .